

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Artificial Intelligence and Machine Learning

ASSESSMENT 2 – Industry Problem Task (5 QUESTIONS)

Course Code:	MLA0407	Course Title:	Deep Learning
CO Assessed:	CO3	Assessment:	Assessment 2: Industry Problem Task
Weightage:	35% of CIA	Total Marks:	(5 Questions × 10 Marks)
CO1 Statement:	<i>Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems.</i>		
Student Name:	Anand paul	Reg. No.:	192425207
Section / Batch:	AIML	Date:	

Code of Conduct

I Anandpaul(192425207) (NAME/ REG NO) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _____

QUESTIONS: 5 Total Marks: 50

Annexure A

Industry Problem Task

1. Industrial Defect Detection

A manufacturing company wants to automatically identify defective products from conveyor belt images. Design a transfer learning-based defect detection/classification solution using DenseNet or another suitable pre-trained CNN. Explain your model selection, feature extraction, fine-tuning strategy, and evaluation metrics.

2. Medical Diagnosis Support System

A hospital wants an AI system to assist doctors in identifying diseases from medical images. Develop a deep learning solution using transfer learning for medical image classification. Compare feature extraction and fine-tuning approaches and justify the final model.

3. Customer Review Sentiment Analysis

An e-commerce company receives thousands of customer reviews every day and wants to automatically understand customer sentiment. Design an RNN/LSTM/BiRNN-based sentiment analysis system to classify reviews as positive, negative, or neutral.

4. Predictive Text for a Mobile Application

A mobile keyboard company wants to provide intelligent next-word suggestions. Develop an LSTM based next-word prediction model and explain how sequential data is processed using BPTT and how LSTM handles long-term dependencies.

5. Automated Customer Support

A multinational company wants to automatically translate customer queries between languages. Design an Encoder–Decoder-based Sequence-to-Sequence model for machine translation. Explain the role of the encoder, decoder, and sequence representation.

6. Social Media Content Monitoring

A social media platform wants to automatically classify posts based on sentiment or category. Develop and compare RNN, BiRNN, and LSTM models for the problem. Analyze which architecture is most suitable for understanding contextual information.

7. Agriculture Crop Disease Detection

An agricultural technology company wants to identify crop diseases from leaf images submitted by farmers. Develop a transfer learning-based image classification system using DenseNet. Apply data augmentation, feature extraction, and fine-tuning to improve performance.

8. Financial News Analysis

A financial organization wants to analyze news headlines and predict whether the market sentiment is positive, negative, or neutral. Design an LSTM or BiRNN-based sequential learning model and compare its performance with a basic RNN.

9. Automated Document Processing

A company receives a large number of scanned documents and wants to automatically classify them into categories such as invoices, resumes, reports, and contracts. Design a deep learning-based document classification system using transfer learning. Discuss preprocessing, feature extraction, model development, and fine-tuning.

10. Smart Traffic Monitoring

A smart city project wants to classify images captured from traffic cameras into categories such as cars, buses, motorcycles, and emergency vehicles. Develop a transfer learning-based traffic classification model using a pre-trained DenseNet architecture and optimize it for real-time d

1. Industrial Defect Detection

Aim:

To automatically identify defective products from conveyor-belt images using Transfer Learning.

Model: DenseNet121 pre-trained on ImageNet.

Method:

1. Collect images of **defective and non-defective products**.
2. Resize images and normalize pixel values.
3. Apply data augmentation such as rotation, flipping and zooming.
4. Use DenseNet as a **feature extractor**.
5. Add a new classification layer for Defective/Non-defective.
6. Initially freeze DenseNet layers and train the classifier.
7. Unfreeze the last few DenseNet layers and fine-tune using a small learning rate.

Evaluation:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Result:

DenseNet can effectively identify product defects because it uses powerful pre-trained visual features. Fine-tuning can further improve performance on manufacturing images.

2. Medical Diagnosis Support System

Aim:

To assist doctors in identifying diseases from medical images using Transfer Learning.

Model: DenseNet121 or ResNet50.

Method:

1. Collect labelled medical images such as X-rays.
2. Resize and normalize the images.
3. Use the pre-trained CNN for feature extraction.
4. Add a custom classification layer.

5. First **freeze the CNN layers** and train only the classifier.
6. Then **unfreeze selected deeper layers** and fine-tune the model.
7. Compare both approaches.

Feature Extraction vs Fine-Tuning

Feature Extraction	Fine-Tuning
CNN layers remain frozen	Some CNN layers are unfrozen
Faster training	More training required
Less computational cost	Higher computational cost
Good for small datasets	Usually better adaptation

Evaluation: Accuracy, Precision, Recall, F1-score and AUC.

Result:

Fine-tuning the final CNN layers is selected if it provides better validation performance without overfitting.

3. Customer Review Sentiment Analysis

Aim:

To classify customer reviews as **Positive, Negative or Neutral**.

Model: LSTM.

Method:

1. Collect customer reviews.
2. Remove unnecessary symbols and convert text to lowercase.
3. Tokenize the reviews.
4. Convert words into numerical sequences.
5. Use an embedding layer.
6. Pass sequences through an LSTM.
7. Use a Softmax output layer with three classes.
8. Train and validate the model.

Architecture:

Review → Tokenization → Embedding → LSTM → Dense → Softmax

Evaluation:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Result:

LSTM is suitable because it can remember important information from earlier words and understand the overall sentiment of a review.

4. Predictive Text for a Mobile Application

Aim:

To predict the next word while a user types on a mobile keyboard.

Model: LSTM.

Example:

Input: "I am going to"

Prediction: "college"

Method:

1. Collect a large text corpus.
2. Tokenize the text into words.
3. Create input-output sequences.
4. Convert words into numerical tokens.
5. Pass sequences through an embedding layer and LSTM.
6. Use Softmax to predict the next word.

BPTT

Backpropagation Through Time (BPTT) calculates the error across different time steps and updates the network weights.

LSTM

LSTM uses **cell state and gates**:

- Forget gate – removes unnecessary information.
- Input gate – stores important new information.

- Output gate – produces relevant output.

Result:

LSTM can learn long-term word dependencies and provide more meaningful next-word suggestions.

5. Automated Customer Support – Machine Translation

Aim:

To automatically translate customer queries from one language to another.

Model: Encoder–Decoder Sequence-to-Sequence model.

Architecture:

Input Sentence → Encoder → Context Representation → Decoder → Translated Sentence

Method:

1. Collect parallel sentences in two languages.
2. Tokenize both languages.
3. Convert words into numerical sequences.
4. Encoder reads the complete input sequence.
5. Encoder creates a sequence representation.
6. Decoder uses this representation to generate the translated sentence.
7. Train using teacher forcing.
8. Evaluate using BLEU score.

Encoder: Understands the input sequence.

Decoder: Generates the output sequence word by word.

Result:

The Encoder–Decoder model can learn relationships between two languages and automatically generate translations.

6. Social Media Content Monitoring

Aim:

To classify social media posts according to sentiment or category.

Models: RNN, BiRNN and LSTM.

Method:

1. Collect social media posts.
2. Clean and tokenize the text.
3. Convert words into embeddings.
4. Train RNN, BiRNN and LSTM separately.
5. Compare their performance.

Comparison

Model	Main Feature
RNN	Processes sequence in one direction
BiRNN	Processes forward and backward
LSTM	Handles long-term dependencies

Result:

BiRNN or LSTM is generally more suitable than a basic RNN. BiRNN can use both previous and future context, while LSTM is better at remembering important information over long sequences.

7. Agriculture Crop Disease Detection

Aim:

To identify crop diseases from leaf images using DenseNet Transfer Learning.

Model: DenseNet121.

Method:

1. Collect healthy and diseased leaf images.
2. Resize and normalize the images.
3. Apply **data augmentation** such as rotation, flipping and zooming.
4. Load pre-trained DenseNet.
5. Freeze DenseNet and use it for feature extraction.
6. Add a custom classification layer.
7. Train the classifier.
8. Unfreeze the last few layers.
9. Fine-tune using a low learning rate.

Architecture:

Leaf Image → DenseNet → Features → Classifier → Disease Class

Evaluation:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Result:

Data augmentation improves generalization, while fine-tuning helps DenseNet adapt to specific crop diseases.

8. Financial News Analysis

Aim:

To classify financial news as **Positive, Negative or Neutral**.

Model: LSTM and basic RNN.

Method:

1. Collect financial news headlines.
2. Clean and tokenize the text.
3. Convert words into embeddings.
4. Train a basic RNN.
5. Train an LSTM using the same dataset.
6. Compare their results.

Architecture:

News Headline → Tokenization → Embedding → RNN/LSTM → Dense → Sentiment

Comparison

Feature	RNN	LSTM
Short-term information	Good	Good
Long-term information	Weak	Strong

Vanishing gradient	More likely	Reduced
Context understanding	Moderate	Better

Result:

LSTM is preferred because financial headlines can contain relationships between words that are separated by several positions.

9. Automated Document Processing

Aim:

To automatically classify scanned documents into **Invoices, Resumes, Reports and Contracts**.

Model: Transfer Learning using ResNet/DenseNet.

Method:

1. Collect labelled document images.
2. Scan or convert documents into images.
3. Resize images to the model's input size.
4. Normalize pixel values.
5. Apply augmentation if required.
6. Use a pre-trained CNN for feature extraction.
7. Add a custom classification layer.
8. Freeze the CNN initially.
9. Fine-tune selected layers for better performance.

Architecture:

Scanned Document → Preprocessing → CNN → Feature Extraction → Classifier → Document Category

Evaluation:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Result:

Transfer learning reduces training time and helps classify different document types effectively,

especially when the available dataset is limited.

10. Smart Traffic Monitoring

Aim:

To classify traffic-camera images into **cars, buses, motorcycles and emergency vehicles**.

Model: Pre-trained DenseNet121.

Method:

1. Collect labelled traffic images.
2. Resize and normalize images.
3. Apply data augmentation.
4. Load pre-trained DenseNet.
5. Remove its original classification layer.
6. Use DenseNet as a feature extractor.
7. Add a custom Softmax classification layer.
8. Freeze the base model and train the classifier.
9. Fine-tune the final DenseNet layers.
10. Optimize the model for fast prediction.

Architecture:

Traffic Image → DenseNet → Feature Extraction → Dense Layer → Softmax → Vehicle Class

Evaluation:

- Accuracy
- Precision
- Recall
- F1-score
- Inference time
- Confusion Matrix

Real-Time Optimization:

- Use smaller input images.
- Freeze unnecessary layers.
- Use batch processing.
- Use GPU acceleration when available.
- Use a lightweight deployment version if required.

Result:

DenseNet Transfer Learning can accurately classify different vehicle types. Fine-tuning improves classification for traffic-specific images, while optimization reduces prediction time for real-time monitoring.